SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

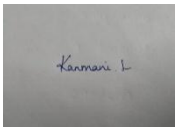
QUESTIONS: **Total Marks: 50**

Annexure A

Code of Conduct:

I _____Kanmani L/192511414_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A
Debugging & Optimisation Task

1. Debugging Hash Function Implementation (SHA Family)
A Python program implementing a SHA-based hashing algorithm produces inconsistent hash outputs for the same input.

Identify errors in message padding and block processing steps.
Fix issues related to bitwise operations and endianness handling.
Optimize the implementation to improve processing speed for large files (≥50 MB).
Evaluate how secure hash functions improve integrity compared to basic checksum methods.

---

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation
A Python-based simulation of SSL/TLS handshake shows delays and occasional handshake failures.

Debug issues in certificate validation and key exchange steps.
Optimize the handshake process to reduce latency.
Refactor the code to reuse session keys efficiently (session resumption).
Analyze the performance vs security trade-offs in TLS optimization.

---

3. Debugging and Optimizing Digital Signature Algorithm (DSA)
A Python implementation of DSA generates invalid signatures during verification.

Identify issues in parameter selection and random number generation.
Fix errors in signature generation and verification steps.
Optimize modular arithmetic operations for faster execution.
Evaluate the impact of randomness quality on signature security.

---

4. Debugging Key Management System
A Python-based key management system fails to securely store and retrieve cryptographic keys.

Identify flaws in key storage (plaintext storage, improper encryption).
Fix issues related to key lifecycle management (generation, distribution, rotation).
Optimize secure storage using encryption and access control mechanisms.
Analyze how poor key management can compromise an entire cryptographic system.

---

5. Optimization of Secure File Transfer Protocol (SFTP-like System)
A Python script simulating secure file transfer experiences slow transfer speeds and occasional data corruption.

Debug issues in encryption/decryption synchronization.
Fix errors in packet handling and integrity verification.
Optimize throughput for large file transfers (≥100 MB).
Compare the optimized system with insecure file transfer methods and justify the improvements.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

1. Debugging Hash Function Implementation (SHA Family)
A Python program implementing a SHA-based hashing algorithm produces inconsistent hash outputs for the same input.

Identify errors in message padding and block processing steps.
Fix issues related to bitwise operations and endianness handling.
Optimize the implementation to improve processing speed for large files (≥50 MB).
Evaluate how secure hash functions improve integrity compared to basic checksum methods.

Answers:

Debugging Hash Function Implementation (SHA Family) (10 Marks)

Introduction

Secure Hash Algorithm (SHA) is a cryptographic hash function that converts input data into a fixed-length hash value. It ensures data integrity by generating a unique digest for every message. Incorrect implementation can produce inconsistent hash outputs for the same input.

1. Errors in Message Padding and Block Processing

SHA-256 processes data in 512-bit (64-byte) blocks, so correct padding is essential. Errors in padding or block division lead to incorrect hash values. Common issues include:

- Incorrect calculation of message length before padding.
- Missing the mandatory '1' bit after the original message.
- Incorrect number of padding zeros.
- Appending the message length using little-endian instead of big-endian format.
- Incorrect division of the message into 512-bit blocks.

2. Fixing Bitwise Operations and Endianness

SHA relies on accurate bitwise operations and proper byte ordering. The implementation should use correct 32-bit rotations and perform all additions modulo 2^{32}.

```
def rotr(x, n):
    return ((x >> n) | (x << (32 - n))) & 0xFFFFFFFF
```

Other corrections include:

- Read input blocks using big-endian byte order.
- Verify logical operations (AND, OR, XOR, NOT) are correctly implemented.

3. Performance Optimization (Files ≥50 MB)

Large files should be processed efficiently to improve execution speed and reduce memory usage.

- Read files in chunks instead of loading the entire file.
- Reuse memory buffers.
- Use Python's optimized hashlib library.
- Reduce unnecessary copying of byte arrays.
- Process blocks sequentially to minimize RAM usage.

4. Evaluation: SHA vs Basic Checksums

SHA Hash Function	Basic Checksum
Cryptographically secure	Not secure
Collision resistant	High collision probability
Detects intentional modifications	Detects only accidental errors
Used in digital signatures	Used only for error detection

Advantages:

- High integrity verification
- Collision resistance
- One-way function
- Widely used in SSL/TLS, Blockchain, and Digital Signatures.

Correct implementation of padding, endianness, and bitwise operations ensures accurate SHA output. Performance optimization improves hashing speed for large files while maintaining strong security and data integrity.

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation
A Python-based simulation of SSL/TLS handshake shows delays and occasional handshake failures.

Debug issues in certificate validation and key exchange steps.
Optimize the handshake process to reduce latency.
Refactor the code to reuse session keys efficiently (session resumption).
Analyze the performance vs security trade-offs in TLS optimization.

Answer:

Secure Socket Layer (SSL) and Transport Layer Security (TLS) are cryptographic protocols used to establish secure communication between a client and a server. A poorly implemented SSL/TLS handshake may result in delays, connection failures, and reduced system performance.

1. Debugging Certificate Validation and Key Exchange

Certificate validation and key exchange are critical steps in the TLS handshake. Errors in these stages can cause handshake failures. Common issues include:

- Expired or invalid digital certificates.
- Hostname mismatch between the certificate and server.
- Untrusted Certificate Authority (CA).
- Incorrect public/private key pairs.
- Errors during Diffie-Hellman or ECDHE key exchange.

2. Optimizing the Handshake Process

The handshake process can be optimized to reduce latency while maintaining security.

- Use **TLS 1.3**, which requires fewer communication rounds.
- Cache validated certificates to avoid repeated verification.
- Reduce unnecessary certificate validation requests.
- Enable efficient cipher suites for faster encryption and decryption.
- Minimize network round trips during the handshake.

3. Session Resumption

Session resumption allows previously established session keys to be reused, eliminating the need for a complete handshake during subsequent connections.

- Use **Session IDs** or **Session Tickets**.
- Reuse previously negotiated session keys securely.
- Reduce CPU usage and handshake time.
- Improve connection speed for repeated client-server communication.

4. Performance vs Security Trade-offs

Optimization	**Performance**	**Security**
TLS 1.3	Faster handshake	Very High
Session Resumption	Reduces latency	Slight risk if sessions remain valid too long
Certificate Caching	Faster validation	Secure if certificates are updated regularly
Smaller Key Size	Faster computation	Lower security strength

Advantages

- Faster and more efficient handshakes.
- Reduced server workload.
- Lower network latency.
- Secure encrypted communication.
- Improved user experience.

Debugging certificate validation and key exchange ensures successful TLS handshakes. Optimizing the handshake process and implementing session resumption significantly improve performance while maintaining strong security. A proper balance between performance and security is essential for reliable and secure communication.

3. Debugging and Optimizing Digital Signature Algorithm (DSA)
A Python implementation of DSA generates invalid signatures during verification.

Identify issues in parameter selection and random number generation.
Fix errors in signature generation and verification steps.
Optimize modular arithmetic operations for faster execution.
Evaluate the impact of randomness quality on signature security.

Answer:

The Digital Signature Algorithm (DSA) is a public-key cryptographic algorithm used to provide authentication, data integrity, and non-repudiation. If a DSA implementation generates invalid signatures during verification, the problem is usually related to incorrect parameter selection, weak random number generation, or errors in the signing and verification process.

1. Issues in Parameter Selection and Random Number Generation

Proper parameter selection is essential for generating valid digital signatures. Common issues include:

- Using invalid or non-prime values for p and q.
- Incorrect selection of the generator g.
- Weak or predictable private keys.
- Reusing the same random number (k) for multiple signatures.
- Using non-cryptographic random number generators.

2. Fixing Signature Generation and Verification

The signature generation and verification steps must strictly follow the DSA algorithm.

- Generate a secure random value k for every signature.

- Compute the signature values r and s correctly using modular arithmetic.
- Verify the public key and message hash before verification.
- Ensure the computed verification value matches the generated signature.
- Reject signatures if r or s falls outside the valid range.

3. Optimizing Modular Arithmetic Operations

Efficient modular arithmetic improves the performance of DSA, especially for large keys.

- Use Python's built-in pow(base, exponent, modulus) for fast modular exponentiation.
- Avoid repeated calculations by reusing intermediate values where possible.
- Use optimized modular inverse algorithms.
- Reduce unnecessary computations during signature verification.

4. Impact of Randomness Quality on Signature Security

Good Randomness	Poor Randomness
Produces secure signatures	May reveal the private key
Prevents signature forgery	Increases risk of forgery
Generates unique random values	Reuses predictable values
Improves overall security	Weakens the cryptographic system

Advantages

- Ensures message authenticity.
- Provides data integrity.
- Supports non-repudiation.
- Enables secure and reliable digital signatures.
- Improves efficiency through optimized computations.

Correct parameter selection, secure random number generation, and accurate implementation of signature generation and verification are essential for a reliable DSA system. Optimizing modular arithmetic improves execution speed, while high-quality randomness ensures strong security and protects against attacks such as private key recovery and signature forger

4. Debugging Key Management System

A Python-based key management system fails to securely store and retrieve cryptographic keys.

Identify flaws in key storage (plaintext storage, improper encryption).
Fix issues related to key lifecycle management (generation, distribution, rotation).

Optimize secure storage using encryption and access control mechanisms. Analyze how poor key management can compromise an entire cryptographic system.

Answer:

A Key Management System (KMS) is responsible for securely generating, storing, distributing, rotating, and destroying cryptographic keys. If a key management system fails to protect keys properly, attackers can gain unauthorized access, making even strong encryption ineffective.

1. Identifying Flaws in Key Storage

Keys must be stored securely to prevent unauthorized access. Common flaws include:

- Storing cryptographic keys in plaintext.
- Using weak or improper encryption methods.
- Hardcoding keys within the source code.
- Lack of authentication and access control.
- Absence of audit logs for key access and usage.

2. Fixing Key Lifecycle Management

A secure key lifecycle ensures that cryptographic keys remain protected throughout their usage.

- Generate keys using cryptographically secure random number generators.
- Distribute keys through secure communication channels such as TLS.
- Rotate encryption keys periodically to reduce the risk of compromise.
- Revoke and replace compromised or expired keys immediately.
- Securely destroy keys that are no longer required.

3. Optimizing Secure Storage

Secure storage can be improved using strong encryption and proper access control mechanisms.

- Encrypt stored keys using AES-256 or a similar strong encryption algorithm.
- Store keys in Hardware Security Modules (HSMs) or secure key vaults.
- Implement Role-Based Access Control (RBAC) to restrict key access.
- Enable multi-factor authentication for administrators.
- Maintain secure backups and audit logs for key management activities.

4. Impact of Poor Key Management

Good Key Management	Poor Key Management
Keys are securely encrypted	Keys stored in plaintext
Strong access control	Unauthorized access

Good Key Management	Poor Key Management
Regular key rotation	Long-term key exposure
Secure key backup	Risk of permanent data loss

Advantages

- Protects sensitive data from unauthorized access.
- Ensures confidentiality and integrity.
- Supports secure authentication.
- Reduces the risk of data breaches.
- Helps organizations comply with security standards.

Effective key management is essential for maintaining the security of any cryptographic system. Secure key storage, proper lifecycle management, strong encryption, and strict access control prevent key compromise and ensure that encrypted data remains protected throughout its lifetime.

5. Optimization of Secure File Transfer Protocol (SFTP-like System)
A Python script simulating secure file transfer experiences slow transfer speeds and occasional data corruption.

Debug issues in encryption/decryption synchronization.
Fix errors in packet handling and integrity verification.
Optimize throughput for large file transfers (≥100 MB).
Compare the optimized system with insecure file transfer methods and justify the improvements.

Answer:

Secure File Transfer Protocol (SFTP) is a secure method of transferring files over a network using encryption and authentication. A poorly implemented SFTP-like system may experience slow transfer speeds, synchronization issues, and data corruption, especially when transferring large files.

1. Debugging Encryption/Decryption Synchronization

Encryption and decryption must remain synchronized between the sender and receiver to ensure correct data transfer. Common issues include:

- Mismatch in encryption and decryption keys.
- Incorrect Initialization Vector (IV) usage.
- Different encryption algorithms or cipher modes.
- Loss of synchronization during data transmission.
- Improper handling of encryption states.

2. Fixing Packet Handling and Integrity Verification

Proper packet handling ensures reliable and error-free file transfer.

- Maintain correct packet sequencing during transmission.
- Detect missing or corrupted packets.
- Use SHA-256 **or** Message Authentication Codes (MACs**)** to verify data integrity.
- Retransmit corrupted or lost packets.
- Validate packet headers before processing.

3. Optimizing Throughput for Large File Transfers (≥100 MB)

Performance can be improved by optimizing the transfer process for large files.

- Transfer files in fixed-size chunks instead of loading the entire file.
- Use buffering to reduce disk I/O operations.
- Compress files before encryption whenever appropriate.
- Enable parallel or multi-threaded data transfer.
- Optimize encryption algorithms and network buffer sizes.

4. Comparison with Insecure File Transfer Methods

Optimized SFTP System	**Insecure File Transfer (FTP)**
Encrypted file transfer	Data transmitted in plaintext
Strong user authentication	Weak or no authentication
Integrity verification using SHA-256/MAC	No integrity protection
Protects against eavesdropping and tampering	Vulnerable to interception and data modification

Advantages

- Ensures confidentiality of transferred files.
- Detects data corruption and unauthorized modifications.
- Provides secure authentication.
- Improves transfer speed for large files.
- Offers reliable and secure communication over public networks.

Optimizing encryption synchronization, packet handling, and throughput significantly improves the performance and reliability of an SFTP-like system. Compared with insecure file transfer methods, the optimized system provides better confidentiality, integrity, authentication, and efficient large-file transfers, making it suitable for secure communication.